

PortFlow

PostgreSQL & Prisma — Problems We Solve & How

Layer	Tool	Role
Database	PostgreSQL	Stores & protects all data
ORM	Prisma	Lets Node.js talk to PostgreSQL easily
Backend	Node.js/Express	Business logic + REST API
Frontend	React	UI that users see & interact with

1. Why PostgreSQL? (Not Just Any Database)

PortFlow manages **Ships, Berths, Cargo, Warehouses, Trucks, Billing** — many related tables that depend on each other. Choosing the wrong database causes *data corruption, duplicates, and race conditions*. PostgreSQL is a **relational database** that prevents all of this.

#	Real Problem in PortFlow	What Goes Wrong Without Fix	PostgreSQL Feature
1	Cargo linked to a ship that doesn't exist	Orphan records, broken queries	Foreign Keys (FK)
2	cargo.quantity = -50	Invalid data in DB	CHECK Constraints
3	Allocate cargo + generate bill (must both succeed)	Cargo added but billing fails → inconsistent state	ACID Transactions
4	Two operators update same cargo at once	Lost updates, data overwrite	MVCC Concurrency
5	Fetch ship → all its cargo → warehouse	Multiple slow queries or wrong joins	JOINS + Indexes
6	Two ships same IMO number	Duplicate rows	UNIQUE Constraint

Problem 1 in Detail — Foreign Key

Problem Cargo record says ship_id = 999, but Ship 999 doesn't exist. Query breaks, app crashes.

Solution PostgreSQL FK rejects the insert. Cargo can only reference a real ship. Consistency guaranteed.

How it looks in the database:

Ships table		Cargo table			
id	name	id	ship_id	item	
101	MV Ocean Star	501	101	Electronics	valid
102	MV Pacific	502	999	Fuel	rejected by FK

Problem 3 in Detail — ACID Transaction

■ Problem

Step 1 (allocate cargo) succeeds. Step 2 (create billing) fails. Now cargo is allocated but client is never billed.

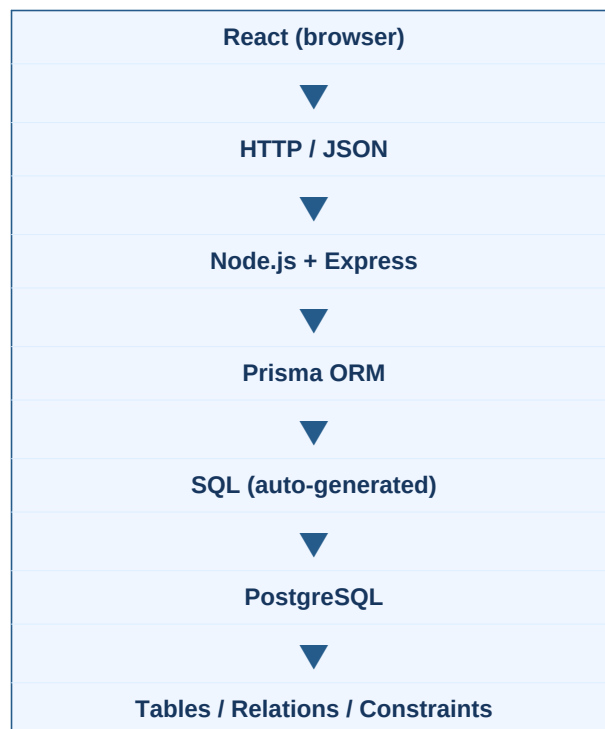
■ Solution

PostgreSQL wraps both in a transaction. If billing fails, cargo allocation is rolled back. Either BOTH succeed or NOTHING changes.

```
BEGIN;  
  INSERT INTO cargo_allocation (ship_id, warehouse_id, qty) VALUES (101, 5, 500);  
  INSERT INTO billing          (cargo_id, amount)          VALUES (501, 25000);  
COMMIT;  -- ← only if BOTH succeeded  
-- if anything fails → ROLLBACK (as if nothing happened)
```

2. Why Prisma ORM? (Not Raw SQL in Node.js)

Prisma sits between **Node.js (Express)** and **PostgreSQL**. It converts JavaScript function calls into the correct SQL automatically.



Problems Prisma Solves

Problem	Without Prisma	With Prisma
Read one ship	<code>db.query("SELECT * FROM ships WHERE id = \$1", [101])</code>	<code>prisma.ship.findUnique({ where: { id: 101 } })</code>
Type safety	Raw query returns plain object, no type hints, easy typos	Returns typed <code>ShipObject</code> in TS, IDE autocomplete works
Related data (ship + cargo)	Write JOIN manually; easy to get wrong	<code>include: { cargo: true }</code> Prisma writes the JOIN
SQL dialects	MySQL syntax ≠ Postgres syntax; migration is painful	Change one line in Prisma config; queries auto-adapt

3. CRUD Operations — Side-by-Side

Every database operation has a plain Prisma equivalent. No manual SQL needed.

READ — Fetch a Ship

■ Raw SQL (manual, error-prone)

```
db.query("SELECT * FROM ships WHERE id = $1", [101]);
```

■ Prisma (clean, typed)

```
const ship = await prisma.ship.findUnique({
  where: { id: 101 }
});
```

Prisma returns a typed JavaScript object:

```
{ id: 101, name: "MV Ocean Star", status: "ARRIVED" }
```

Node.js sends this to React as JSON — frontend displays it instantly.

CREATE — Add Cargo

■ Raw SQL

```
db.query(
  "INSERT INTO cargo (ship_id, warehouse_id, quantity)
  + " VALUES ($1,$2,$3)", [101, 5, 500]);
```

■ Prisma

```
const cargo = await prisma.cargo.create({
  data: {
    shipId: 101,
    warehouseId: 5,
    quantity: 500
  }
});
```

UPDATE — Change Ship Status

■ Raw SQL

```
db.query("UPDATE ships SET status=$1 WHERE id=$2",
  ["DEPARTED", 101]);
```

■ Prisma

```
await prisma.ship.update({
  where: { id: 101 },
  data: { status: "DEPARTED" }
});
```

DELETE — Remove Cargo

■ Raw SQL

```
db.query("DELETE FROM cargo WHERE id=$1", [501]);
```

■ Prisma

```
await prisma.cargo.delete({
  where: { id: 501 }
});
```

■ Prisma maps: CREATE → .create() READ → .findMany() UPDATE → .update() DELETE → .delete()

4. Fetching Related Data (The JOIN Problem)

■ Problem

To show 'Ship + all its Cargo' you need a JOIN. Writing JOINS manually is verbose and easy to break when schema changes.

■ Solution

Prisma's 'include' keyword generates the JOIN for you. One function call, zero manual SQL.

Ship 101 → **Cargo 501** (Electronics, qty 500, Warehouse 5)
→ **Cargo 502** (Steel, qty 200, Warehouse 3)

Prisma query:

```
const ship = await prisma.ship.findUnique({
  where: { id: 101 },
  include: { cargo: true } // ← Prisma writes the JOIN
});
```

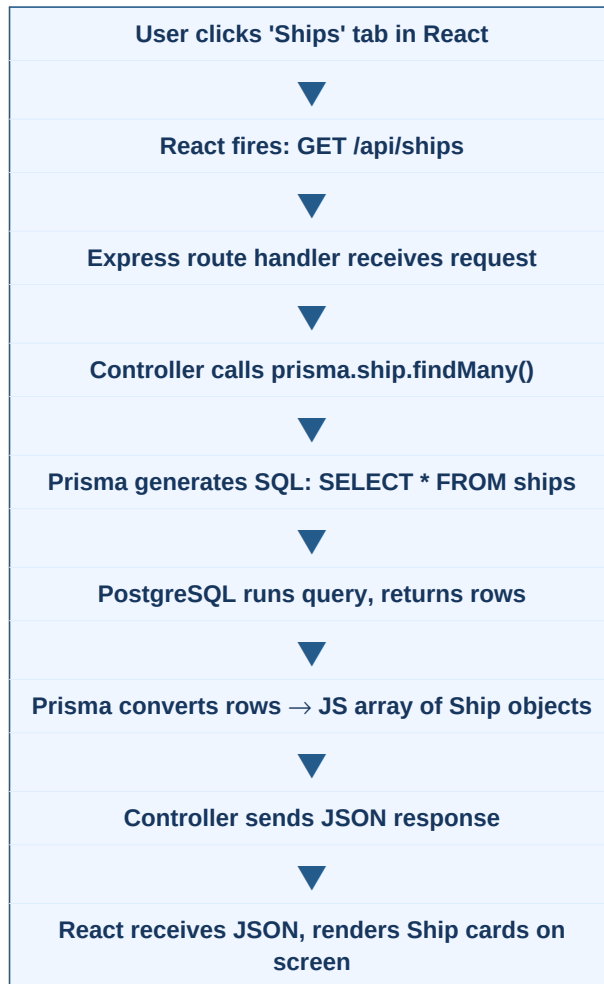
Result returned to Node.js (then forwarded to React as JSON):

```
{
  "id": 101,
```

```
"name": "MV Ocean Star",
"status": "ARRIVED",
"cargo": [
  { "id": 501, "item": "Electronics", "quantity": 500, "warehouseId": 5 },
  { "id": 502, "item": "Steel", "quantity": 200, "warehouseId": 3 }
]
```

5. Full Request Flow — Ships Page

When a user opens the Ships page, here is the **exact data journey** from click to screen:



Sample API Response:

```
[
  { "id": 101, "name": "MV Ocean Star", "status": "ARRIVED" },
  { "id": 102, "name": "MV Pacific", "status": "DEPARTED" }
]
```

Summary — What Each Tool Does

Tool	What it does	Without it
PostgreSQL	Stores data in structured tables. Enforces FKs, constraints, ACID transactions.	Data corruption, duplicates, race conditions
Prisma ORM	Translates JS code into SQL. Returns typed objects to Node.js.	Manual SQL everywhere, easy typos, no type safety

Tool	What it does	Without it
Node.js/Express	Runs business logic. Exposes REST API to React.	No server, no API, frontend can't get data
React	Displays data to the user. Sends requests (GET/POST/PUT/DELETE).	No UI

■ **Key Rule: Prisma does NOT store data. PostgreSQL stores data. Prisma makes accessing it from Node.js clean and safe.**

One-Line Architecture

```

React
  ↓ HTTP / JSON
Express (routes, controllers)
  ↓ prisma.model.operation()
Prisma ORM (generates SQL)
  ↓ SQL
PostgreSQL (tables, FK, constraints, transactions)

```